

# ROS2 day4 hw2 레포트

로봇 예비단원 주호진

## 1. 과제

공식 문서 참고하여 Lifecycle node + Qos 설정한 publisher, subscriber 패키지 구현

=> Lifecycle 상태 전환을 자동화하는 Launch 파일과 수동으로 제어하는 launch 파일 각각 작성

- 어떤 QoS Policy를 왜 선택했는가
- 각 Callback에서 무엇을 준비·시작·중지·정리했는가
- Active가 아닐 때 실제 Publish / Subscribe가 어떻게 달라지는가

<https://docs.ros.org/en/jazzy/p/lifecycle/>

<https://docs.ros.org/en/jazzy/Tutorials/Demos/Managed-Nodes.html>

## 2. 코드

예제코드 lifecycle\_listener, lifecycle\_talker를 가져와서 보면서 옮겨보았다.

사실상 코드는 파일을 보고 만들었기 때문에 거의 동일하다. 옮기면서 이해와 헤더파일, 소스파일로 나누는 작업을 했다.

**-Publisher**

```

using namespace std::chrono_literals;
using CallbackReturn = rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;

class Day4Hw2Publisher : public rclcpp_lifecycle::LifecycleNode
{
public:
    explicit Day4Hw2Publisher(const std::string &node_name, bool intra_process_comms = true)
        : rclcpp_lifecycle::LifecycleNode(node_name, rclcpp::NodeOptions().use_intra_process_comms(intra_process_comms))
    {
    }

    void publish();

    CallbackReturn on_configure(const rclcpp_lifecycle::State &);
    CallbackReturn on_activate(const rclcpp_lifecycle::State &state);
    CallbackReturn on_deactivate(const rclcpp_lifecycle::State &state);
    CallbackReturn on_cleanup(const rclcpp_lifecycle::State &);
    CallbackReturn on_shutdown(const rclcpp_lifecycle::State &state);

private:
    std::shared_ptr<rclcpp_lifecycle::LifecyclePublisher<std_msgs::msg::String>> pub_;
    std::shared_ptr<rclcpp::TimerBase> timer_;
};

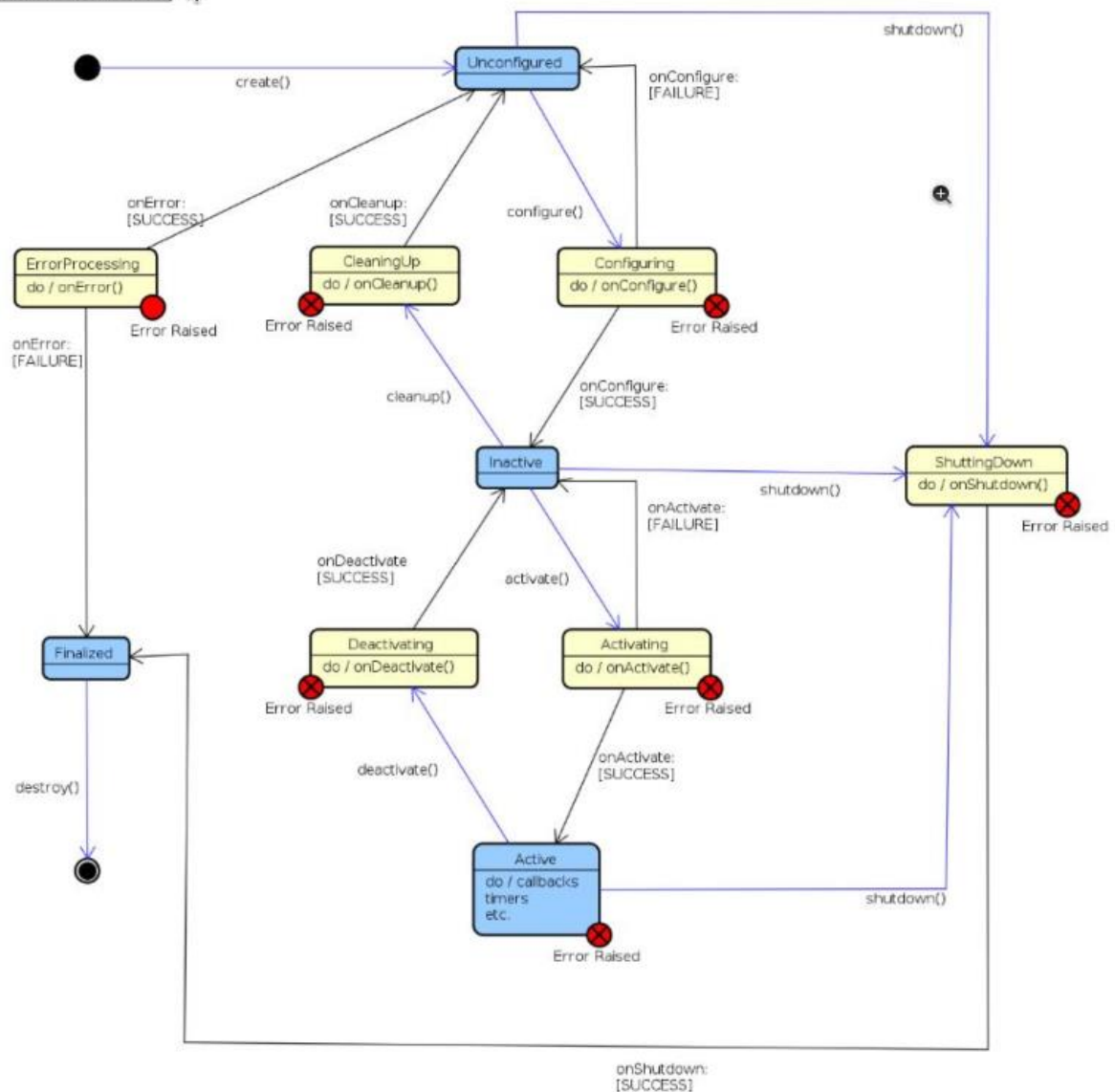
```

`rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn`

일단 콜백 함수 앞에 붙는 자료형 이름이 너무 길어서 `CallbackReturn`으로 줄여서 적용하였다.

Lifecycle에서의 노드는 create되어도 아무것도 구성되면 안되기 때문에 생성자에는 아무 코드도 들어가지 않는다.

헤더 파일에는 전환 콜백 함수와 `publish` 함수가 선언되어있다.



```

// ROS2에서 lifecycle 노드의 콜백을 읽고 publish를 자동으로 열고 닫아준다.
// active인 경우 publish됨, inactive인 경우 publish되지만 미들웨어 계층에서 버려진다고 한다.
// ++count 부분은 계속해서 올라가면서 pub을 보내지만 버려지므로 sub에서 볼 수는 없음
pub_ -> publish(std::move(msg));

```

Pub의 경우 Unconfigured에서 inactive 상태가 되면 pub을 계속 하지만 ROS2에서 라이프사이클의 상태를 확인해서 active 경우 퍼블리시하도록 하고, inactive인 경우 미들웨어 계층에서 버려진다고 한다. (따로 코드에서 구현 X)

```

// on_configure: Unconfigured 상태 -> inactive 상태로
// QoS 설정, 퍼블리셔 생성, 타이머 생성
CallbackReturn Day4Hw2Publisher::on_configure(const

```

```

rclcpp::QoS qos_profile(10); // depth 10
qos_profile.history(rclcpp::HistoryPolicy::KeepLast); // history: keep last
qos_profile.reliability(rclcpp::ReliabilityPolicy::Reliable); // reliability: reliable
qos_profile.durability(rclcpp::DurabilityPolicy::TransientLocal); // durability: volatile

// qos 설정은 위와 같이 history: keep last, depth: 10
// reliability: reliable, durability: TransientLocal로 설정하였다.
// 일반적인 통신을 위한 node라고 가정하고, 통신에서 자주 쓰는 QoS의 설정값을 참고한 뒤 설정을 일부 변경했다.

// 사용 이유
// history: 최근 데이터 유실을 방지하기 위해 몇개의 값을 저장하나, 너무 많은 메모리를 사용하지 않기 위해 10개로 설정
// reliability: 일반적인 통신에서 데이터가 유실되는 것이 치명적일 수도 있기 때문에 유실되지 않도록 reliable로 설정
// durability: 일반적으로 통신에서 durability에 사용되는 값은 volatile인데 TransientLocal로 설정을 해보았다.
// publisher가 상태가 변경되는 경우에 유실되는 데이터를 저장하여 구독할 수 있도록 설정했다.

pub_ = this->create_publisher<std_msgs::msg::String>("lc_topic", qos_profile);
timer_ = this->create_wall_timer(1s, [this]() { return this->publish(); });

```

configure에서는 Unconfigured에서 inactive 상태로 변경하도록 한다.

이때 QoS 설정과 퍼블리셔를 생성한다.

```

// on_activate: inactive에서 active로
// Node 활성화, 데이터 전송 시작
CallbackReturn Day4Hw2Publisher::on_activate(
{
// 부모 클래스 메서드는 관리되는 엔티티(현재)
LifecycleNode::on_activate(state);

```

Activate에서는 inactive상태의 노드를 active로 바꾼다. Activate 상태가 되면 ROS2에서 퍼블리셔를 성공적으로 보내게 된다.

```

// on_deactivate: active에서 inactive로
// Node 활성화 off, 데이터 전송 중지
CallbackReturn Day4Hw2Publisher::on_deactivate(
{
LifecycleNode::on_deactivate(state);

```

Deactivate는 active상태를 inactive로 바꾼다. 이때 노드는 살아있기 때문에 코드 상에서 퍼블리셔는 보내지만 ROS2에서 lifecycle의 inactive 상태를 읽고 퍼블리시되지 않게 처리한다.

```
// on_cleanup: inactivated에서 unconfigured로
// 타이머와 퍼블리셔 해제, on_configure할 수 있는 상태로 돌아감
CallbackReturn Day4Hw2Publisher::on_cleanup(const rclcpp::Context& context)
{
    // 정리 단계에서 타이머와 퍼블리셔에 대한 공유 포인터를 해제함
    timer_.reset();
    pub_.reset();
}
```

cleanup은 퍼블리셔를 해제하고 unconfigured 상태로 되돌린다. Shutdown과 차이점은 다시 퍼블리셔를 생성하고 활성화시킬 수 있다는 점이다.

```
// on_shutdown: 아무런 어떠한 상태에서 Finalized로
// 타이머, 퍼블리셔 해제, 노드를 종료하도록 함
CallbackReturn Day4Hw2Publisher::on_shutdown(const rclcpp::Context& context)
{
    // 종료 단계에서 타이머와 퍼블리셔에 대한 공유 포인터를 해제함
    timer_.reset();
    pub_.reset();
}
```

퍼블리셔를 해제하고 노드를 종료상태로 들어가도록 한다.

## -Subscriber

```
* 두 개의 토픽을 구독합니다.
* - lifecycle_chatter: talker로부터 오는 데이터 토픽
* - lc_talker_transition_event: lc_talker 노드의 상태 변화에 대한 알림을 퍼블리시하는 토픽
*/
class Day4Hw2Subscriber : public rclcpp::Node
{
public:
    explicit Day4Hw2Subscriber(const std::string &node_name);
    void data_callback(std_msgs::msg::String::ConstSharedPtr msg);
    void notification_callback(lifecycle_msgs::msg::TransitionEvent::ConstSharedPtr msg);
private:
    std::shared_ptr<rclcpp::Subscription<std_msgs::msg::String>> sub_data_;
    std::shared_ptr<rclcpp::Subscription<lifecycle_msgs::msg::TransitionEvent>> sub_notification_;
};
```

퍼블리셔에서 보내주는 msg를 subscribe하고, transition\_event라는 상태 변화를 publish 해주는 토픽을 subscribe해서 상태를 터미널에 출력한다.

```

Day4Hw2Subscriber::Day4Hw2Subscriber(const std::string &node_name) : Node(
{
    // lc_talker 노드로부터 오는 데이터 토픽
    sub_data_ = this->create_subscription<std_msgs::msg::String>(
        "lc_topic", 10, [this](std_msgs::msg::String::ConstSharedPtr msg)
    // 알림 이벤트 토픽. 모든 상태 변화는 전환을 나타내는 시작 상태와 목표 상태를 가진 Tr
    // 퍼블리시됩니다.
    sub_notification_ = this->create_subscription<lifecycle_msgs::msg::Tra
        "/lc_pub/transition_event", 10,
        [this](lifecycle_msgs::msg::TransitionEvent::ConstSharedPtr msg) {
    }
}

```

- 런치 파일

```

from launch import LaunchDescription
from launch_ros.actions import LifecycleNode, Node

def generate_launch_description():
    publisher_node = LifecycleNode(
        package='day4_hw2_package',
        executable='publisher_node',
        name='lc_pub',
        namespace='',
        output='screen'
    )

    subscriber_node = Node(
        package='day4_hw2_package',
        executable='subscriber_node',
        name='lc_sub',
        output='screen'
    )

    return LaunchDescription([
        publisher_node,
        subscriber_node,
    ])

```

수동 런치 파일은 다음과 같이 노드만 넣었고, 자동 런치 파일은 지정한 초에 따라 명령어를 넣어주도록 했다.

```

configure_action = TimerAction(
    period=5.0,
    actions=[
        ExecuteProcess(
            cmd=['ros2', 'lifecycle', 'set', '/lc_pub', 'configure'],
            output='screen'
        )
    ]
)

return LaunchDescription([
    publisher_node,
    subscriber_node,
    configure_action,
    activate_action,
    deactivate_action,
    cleanup_action,
    shutdown_action
])

```

### 3. 작동 영상

## 작동 영상

```
hojin@hojins: ~/colcon_ws
colcon build [1/1 done] [0 ongoing] 49x42
d 952048]
[INFO] [ros2-7]: process started with pid [952087]
[publisher_node-1] [INFO] [1789321898.743654636]
[lc_pub]: on shutdown
[subscriber_node-2] [INFO] [1789321898.743809196]
[lc_sub]: notify callback: Transition from state
unconfigured to shuttingdown
[subscriber_node-2] [INFO] [1789321898.743853332]
[lc_sub]: notify callback: Transition from state
shuttingdown to finalized
[ros2-7] Transitioning successful
[INFO] [ros2-7]: process has finished cleanly [pi
d 952087]
^C[WARNING] [launch]: user interrupted with ctrl-
c (SIGINT)
[publisher_node-1] [INFO] [1789321903.145523563]
[rcldcpp]: signal_handler(SIGINT/SIGTERM)
[INFO] [subscriber_node-2]: process has finished
cleanly [pid 951801]
[INFO] [publisher_node-1]: process has finished c
leanly [pid 951800]
[subscriber_node-2] [INFO] [1789321903.145523541]
[rcldcpp]: signal_handler(SIGINT/SIGTERM)
hojin@hojins:~/colcon_ws$ cd ~/colcon_ws
colcon build --packages-select day4_hw2_package
source install/setup.bash
ros2 launch day4_hw2_package manual_launch.py
Starting >>> day4_hw2_package
Finished <<< day4_hw2_package [0.10s]

Summary: 1 package finished [0.24s]
[INFO] [launch]: All log files can be found below
/home/hojin/.ros/log/2026-09-14-02-51-47-959286-
hojins-952166
[INFO] [launch]: Default logging verbosity is set
to INFO
[INFO] [publisher_node-1]: process started with p
id [952182]
[INFO] [subscriber_node-2]: process started with

ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: unconfigured [1]
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: inactive [2]
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: active [3]
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: inactive [2]
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: unconfigured [1]
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: finalized [4]
hojin@hojins:~/colcon_ws$ ^C
hojin@hojins:~/colcon_ws$ ^C
hojin@hojins:~/colcon_ws$
```

## 자동

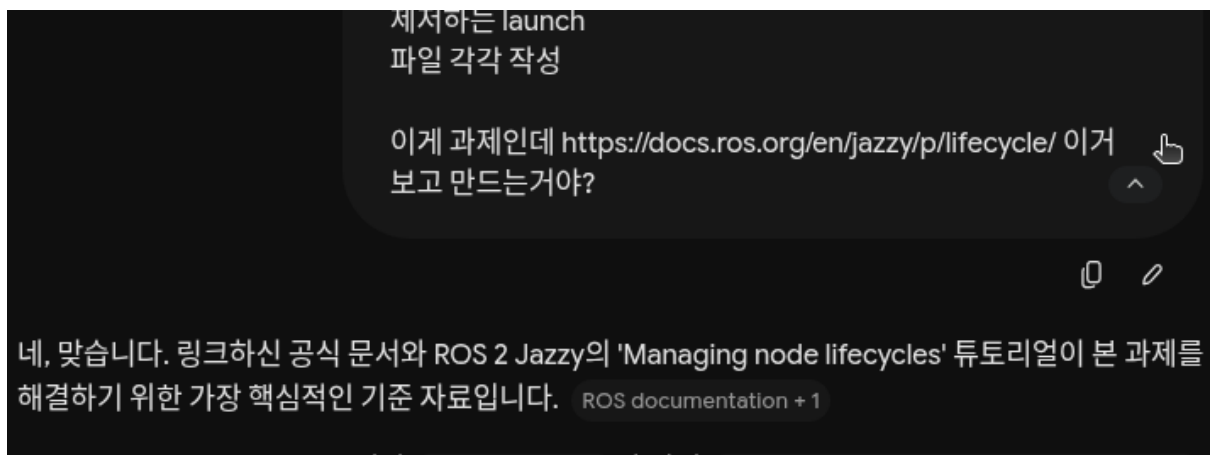
```
hojin@hojins: ~/colcon_ws
colcon build [1/1 done] [0 ongoing] 49x42
[subscriber_node-2] [INFO] [1789321959.320246657]
[lc_sub]: data_callback: Lifecycle HelloWorld #4
3
[publisher_node-1] [INFO] [1789321960.319733839]
[lc_pub]: active. Publishing: [Lifecycle HelloWorld
#44]
[subscriber_node-2] [INFO] [1789321960.320207946]
[lc_sub]: data_callback: Lifecycle HelloWorld #4
4
[publisher_node-1] [INFO] [1789321961.319756295]
[lc_pub]: active. Publishing: [Lifecycle HelloWorld
#45]
[subscriber_node-2] [INFO] [1789321961.320311169]
[lc_sub]: data_callback: Lifecycle HelloWorld #4
5
[publisher_node-1] [INFO] [1789321961.752042905]
[lc_pub]: on_deactivate
[subscriber_node-2] [INFO] [1789321961.752195589]
[lc_sub]: notify callback: Transition from state
active to deactivating
[subscriber_node-2] [INFO] [1789321961.752258531]
[lc_sub]: notify callback: Transition from state
deactivating to inactive
[publisher_node-1] [INFO] [1789321962.320064230]
[lc_pub]: currently inactive. Messages are not pu
blished.
[publisher_node-1] [WARN] [1789321962.320345142]
[LifecyclePublisher]: Trying to publish message o
n the topic '/lc_topic', but the publisher is not
activated
[publisher_node-1] [INFO] [1789321963.319929029]
[lc_pub]: currently inactive. Messages are not pu
blished.
[subscriber_node-2] [INFO] [1789321963.525356595]
[lc_sub]: notify callback: Transition from state
inactive to cleanup
[publisher_node-1] [INFO] [1789321963.527140731]
[lc_pub]: on_cleanup
[subscriber_node-2] [INFO] [1789321963.528621360]
[lc_sub]: notify callback: Transition from state

/lc_pub: inactive [2]
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: unconfigured [1]
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_pub
lifecycle get /lc_pub: finalized [4]
hojin@hojins:~/colcon_ws$ ^C
hojin@hojins:~/colcon_ws$ ^C
hojin@hojins:~/colcon_ws$ ros2 lifecycle set /lc_p
ub configure
Transitioning successful
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_p
ub lifecycle get /lc_pub: inactive [2]
hojin@hojins:~/colcon_ws$ ros2 lifecycle set /lc_p
ub activate
Transitioning successful
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_p
ub lifecycle get /lc_pub: active [3]
hojin@hojins:~/colcon_ws$ ros2 lifecycle set /lc_p
ub deactivate
Transitioning successful
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_p
ub lifecycle get /lc_pub: inactive [2]
hojin@hojins:~/colcon_ws$ ros2 lifecycle set /lc_p
ub activate
Transitioning successful
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_p
ub lifecycle get /lc_pub: active [3]
hojin@hojins:~/colcon_ws$ ros2 lifecycle set /lc_p
ub cleanup
Unknown transition requested, available ones are:
- deactivate [4]
- shutdown [7]
hojin@hojins:~/colcon_ws$ ros2 lifecycle set /lc_p
ub deactivate
Transitioning successful
hojin@hojins:~/colcon_ws$ ros2 lifecycle set /lc_p
ub cleanup
Transitioning successful
hojin@hojins:~/colcon_ws$ ros2 lifecycle get /lc_p
ub lifecycle get /lc_pub: unconfigured [1]
```

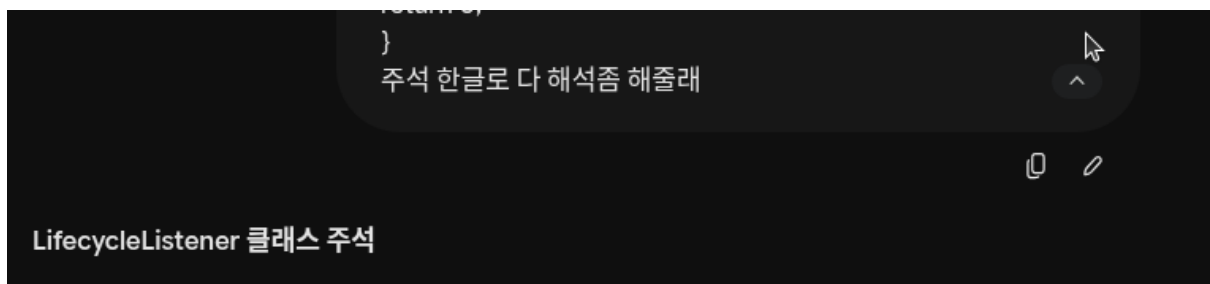
수동

#### 4. AI 사용 보고서

<https://share.gemini.google/LpecSvPdmXg1>



ROS2 lifecycle 관련 자료 사이트를 찾았는데 이게 공식 자료가 맞는 지 물어보았다. 해당 사이트에서 예제 코드는 git에 올라가 있었는데 해당 깃에 lifecycle 외에 다양한 예제 코드가 있었다.



참고 사이트에 나와있는 예제 코드가 깃에 있었는데 영어 주석이 너무 많아서 한글로 바꿔서 달라고 부탁하였다.

## 라이프사이클 노드 명령어 수동으로 전달하는법

### ROS 2 Lifecycle CLI 명령어 활용

#### 실행 중인 라이프사이클 노드 목록 확인

현재 시스템에서 라이프사이클 관리 대상인 노드들을 확인합니다.

Bash

```
ros2 lifecycle nodes
```

과제에 수동으로 제어하는 조건이 있어서 터미널로 어떻게 명령을 전달하는 지 물어봤다.

ros2 lifecycle set 을 하고 /publish\_node이름 뒤에 configure, activate, deactivate, cleanup, shutdown 중 하나를 입력하면 된다.

ros2 lifecycle get을 하면 현재 상태를 확인할 수 있다.

transition\_event은 life cycle내에 있는 기능인가?

네, 맞습니다. `transition_event` 는 ROS 2 라이프사이클(Lifecycle) 프레임워크에 기본적으로 내장된 핵심 기능입니다.

- **자동 퍼블리시 토픽:** 라이프사이클 노드가 생성될 때 자동으로 활성화되며, 노드의 상태가 바뀔 때마다 `<node_name>/transition_event` 토픽으로 상태 변화 알림을 스스로 퍼블리시합니다.

transition\_event에서 노드의 상태 변화를 pub해주는 topic이 있어 이를 사용하면 현재 상태를 읽어들 수 있다.